CSE563 — Unit IV Numerical Worksheet

Graph Theory

40 questions • CO4 • Solutions Booklet

Instructor Note

Full worked solutions for all 40 questions, with step tables shown in the form students are expected to produce. Tie-breaks are alphabetical/numerical-lowest throughout and are stated wherever they affect the answer.

A. Graph Basics and Types (Q1–Q10)

Q1. A graph has degree sequence $(4, 4, 4, 3, 3, 2)$. Find the number of edges. Is such a graph possible?

Solution. Degree sum $= 4+4+4+3+3+2 = 20$, so $|E| = 20/2 = \mathbf{10}$. The sum is even and there are exactly two odd-degree vertices (the two 3s) — an even count, as required. No degree exceeds $n-1 = 5$. **Such a graph is possible.**

Q2. Determine whether a simple graph can exist with degree sequence $(5, 4, 3, 2, 1, 1)$.

Solution. Sum $= 16$, even, so $|E| = 8$; and the odd-degree vertices are $5, 3, 1, 1$ — four of them, an even count. Max degree $5 = n-1$ is attainable on 6 vertices. The handshaking conditions are satisfied, so **no contradiction arises** and such a graph can exist. (For example: the degree-5 vertex joins all others; then add edges among the rest to reach the remaining degrees.)

Q3. How many edges does the complete graph K_8 have, and what is the degree of each vertex?

Solution. $|E| = \binom{8}{2} = \dfrac{8 \times 7}{2} = \mathbf{28}$. Each vertex is adjacent to all others, so every degree is $n-1 = \mathbf{7}$. *Check:* $8 \times 7/2 = 28$ ✓.

Q4. How many edges does the complete bipartite graph $K_{4,6}$ have? Is it bipartite? Is it complete?

Solution. $|E| = 4 \times 6 = \mathbf{24}$. It is **bipartite** by construction (parts of size 4 and 6, all edges crossing between them). It is **not** a complete graph: vertices within the same part are not adjacent. K_{10} would have 45 edges.

Q5. A tree has 15 vertices. How many edges does it have? How many edges must be added to make it contain exactly 3 cycles?

Solution. A tree on n vertices has $n-1$ edges: **14**. Each edge added to a tree creates exactly one new cycle, so to obtain 3 cycles add **3** edges (giving 17 edges in total).

Q6. Write the adjacency matrix for the graph with $V = \{1, 2, 3, 4\}$ and $E = \{12, 13, 23, 34\}$, and compute the degree of each vertex from it.

Solution.

$$A = \begin{pmatrix} 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Degrees are the row sums: $\deg(1) = 2$, $\deg(2) = 2$, $\deg(3) = \mathbf{3}$, $\deg(4) = 1$. *Check:* sum $= 8 = 2|E| = 2(4)$ ✓. The matrix is symmetric, as it must be for an undirected graph.

Q7. For the path graph $1 - 2 - 3 - 4$, compute A^2 and state the number of walks of length 2 from vertex 1 to vertex 3.

Solution.

$$A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \qquad A^2 = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 2 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 1 \end{pmatrix}.$$

$(A^2)_{13} = \mathbf{1}$ — exactly one walk of length 2 from 1 to 3, namely $1 \to 2 \to 3$. $(A^2)_{22} = 2$: the closed walks $2 \to 1 \to 2$ and $2 \to 3 \to 2$.

Q8. Determine whether the graph with edges AB, BC, CD, DA (a 4-cycle) is Eulerian, and whether it is Hamiltonian.

Solution. Degrees: all four vertices have degree 2 — **all even**, and the graph is connected. **Eulerian circuit exists:** $A \to B \to C \to D \to A$ uses every edge exactly once and returns to the start. **Hamiltonian:** the same cycle visits every vertex exactly once — **yes**. (For a cycle graph the Euler circuit and Hamiltonian cycle coincide.)

Q9. A connected graph has degrees $3, 3, 2, 2, 2$. Does it have an Euler circuit, an Euler path, or neither?

Solution. Two vertices have odd degree (the two 3s). **No Euler circuit** (that requires all degrees even), but **an Euler path exists**, beginning at one odd-degree vertex and ending at the other. Edge count: $\sum \deg = 12$, so $|E| = 6$; the path uses all 6 edges.

Q10. Determine whether $K_{3,3}$ is planar, using Euler's inequality for bipartite planar graphs, $m \leq 2n - 4$.

Solution. $K_{3,3}$ has $n = 6$ vertices and $m = 3 \times 3 = 9$ edges. For a bipartite planar graph, $m \leq 2n - 4 = 2(6) - 4 = 8$. Since $9 > 8$, $K_{3,3}$ **is not planar**. (It is one of the two Kuratowski obstructions, along with K_5; the bipartite bound is tighter than the general $m \leq 3n - 6 = 12$, which $K_{3,3}$ would have satisfied.)

B. Traversal: BFS, DFS and Topological Sort (Q11–Q18)

Q11. For the graph with adjacency lists 1:[2, 3], 2:[1, 4, 5], 3:[1, 5], 4:[2, 6], 5:[2, 3, 6], 6:[4, 5], perform BFS from vertex 1. Give the visit order and all distances.

Solution.

Dequeue	Newly visited	Queue after
1	2, 3	$[2, 3]$
2	4, 5	$[3, 4, 5]$
3	— (5 already seen)	$[4, 5]$
4	6	$[5, 6]$
5	—	$[6]$
6	—	$[\,]$

Order: 1, 2, 3, 4, 5, 6. Distances: $d(1) = 0$; $d(2) = d(3) = 1$; $d(4) = d(5) = 2$; $d(6) = 3$.

Q12. For the same graph, perform DFS from vertex 1 (recursive, neighbours in listed order) and give the visit order and the tree edges.

Solution. $1 \to 2$ (first neighbour) $\to 4$ (2's list: 1 seen, then 4) $\to 6$ (4's: 2 seen, then 6) $\to 5$ (6's: 4 seen, then 5) $\to 3$ (5's: 2 seen, then 3). 3's neighbours 1 and 5 are both visited; backtrack throughout.
Order: 1, 2, 4, 6, 5, 3. Tree edges: 1-2, 2-4, 4-6, 6-5, 5-3 — five edges, a spanning tree ✓.
Back edges: 5-2 and 3-1, each proving a cycle exists.

Q13. Using the BFS result of Q11, give the BFS tree edges and verify that the tree path to vertex 6 has length equal to $d(6)$.

Solution. **BFS tree edges:** 1-2, 1-3, 2-4, 2-5, 4-6. Tree path to 6: $1 \to 2 \to 4 \to 6$, of length **3**, which equals $d(6) = 3$ ✓. This illustrates the defining property of BFS: in an unweighted graph the BFS tree path is a shortest path.

Q14. Use BFS to determine whether the graph of Q11 is bipartite.

Solution. Colour by BFS level parity: level 0 = $\{1\}$, level 1 = $\{2, 3\}$, level 2 = $\{4, 5\}$, level 3 = $\{6\}$. Assign colour X to even levels $\{1, 4, 5\}$ and colour Y to odd levels $\{2, 3, 6\}$.
Check every edge: 12 (X,Y) ✓; 13 (X,Y) ✓; 24 (Y,X) ✓; 25 (Y,X) ✓; 35 (Y,X) ✓; 46 (X,Y) ✓; 56 (X,Y) ✓.
No edge joins two vertices of the same parity, so the graph is **bipartite**, with parts $\{1, 4, 5\}$ and $\{2, 3, 6\}$.

Q15. Determine whether the 5-cycle $A - B - C - D - E - A$ is bipartite, justifying your answer two different ways.

Solution. **Way 1 — BFS parity.** From A: level 0 = $\{A\}$, level 1 = $\{B, E\}$, level 2 = $\{C, D\}$. The edge CD joins two level-2 vertices — same parity. **Not bipartite.**
Way 2 — odd cycle. A graph is bipartite iff it contains no odd cycle. This graph *is* a cycle of length 5, which is odd. **Not bipartite.**
Consequently $\chi = 3$, not 2.

Q16. Perform a topological sort of the DAG with edges $A \to B$, $A \to C$, $B \to D$, $C \to D$, $D \to E$ using Kahn's algorithm.

Solution. In-degrees: A=0, B=1, C=1, D=2, E=1.

Step	Available (in-degree 0)	Output
1	A	A
2	B, C	B (alphabetical)
3	C	C
4	D	D
5	E	E

Order: A, B, C, D, E. *Verification:* every edge points forward — $A \to B$ ✓, $A \to C$ ✓, $B \to D$ ✓, $C \to D$ ✓, $D \to E$ ✓. (A, C, B, D, E is an equally valid answer.)

Q17. Course prerequisites: Maths → Physics, Maths → Programming, Physics → Mechanics, Programming → DataStructures, DataStructures → Algorithms, Mechanics → Robotics, Algorithms → Robotics. Give a valid study order.

Solution. In-degree 0 initially: Maths only.

Step	Output
1	Maths
2	Physics (alphabetical among Physics, Programming)
3	Programming
4	Mechanics
5	DataStructures
6	Algorithms
7	Robotics

Valid order: Maths, Physics, Programming, Mechanics, DataStructures, Algorithms, Robotics. Robotics correctly comes last — it requires both Mechanics and Algorithms.

Q18. Explain, using a DFS trace, how a back edge proves the presence of a cycle in the graph with edges $A \to B$, $B \to C$, $C \to A$.

Solution. DFS from A: visit A (on the recursion stack), then B (stack: A, B), then C (stack: A, B, C). From C the edge $C \to A$ leads to A, which is **already visited and still on the recursion stack** — i.e. A is an *ancestor* of C in the DFS tree. That makes $C \to A$ a **back edge**.
A back edge from C to its ancestor A means there is a tree path $A \to B \to C$ *and* a direct edge $C \to A$, which together close the cycle $A \to B \to C \to A$.
This is exactly how cycle detection is implemented: a graph is acyclic iff a DFS finds no back edge, which is why the same traversal also yields a topological sort.

C. Dijkstra and Minimum Spanning Trees (Q19–Q28)

Q19. Run Dijkstra from S on the weighted graph with edges SA=1, SB=4, AB=2, AC=6, BC=3, CD=2, BD=7. Give the table and all final distances.

Solution.

Extract	S	A	B	C	D	Relaxations
init	**0**	∞	∞	∞	∞	—
S (0)	**0**	1	4	∞	∞	A:1, B:4
A (1)	0	**1**	3	7	∞	B:1+2 = 3 < 4 ✓; C:1+6 = 7
B (3)	0	1	**3**	6	10	C:3+3 = 6 < 7 ✓; D:3+7 = 10
C (6)	0	1	3	**6**	8	D:6+2 = 8 < 10 ✓
D (8)	0	1	3	6	**8**	—

Distances: S=0, A=1, B=3, C=6, D=8.

Q20. For the previous question, give the shortest path from S to D and its cost.

Solution. Trace parents backwards: D was last improved from C $(6+2)$; C from B $(3+3)$; B from A $(1+2)$; A from S.

$$\boxed{S \to A \to B \to C \to D, \qquad \text{cost } 1+2+3+2=8.}$$

Check against the direct-ish route: $S \to B \to D = 4+7 = 11$, worse ✓.

Q21. Explain why Dijkstra's algorithm gives the wrong answer on the graph $S \to A$ (weight 2), $S \to B$ (weight 6), $B \to A$ (weight -5). Give the correct distances.

Solution. Dijkstra extracts A first with $d(A) = 2$ and declares it **final**, because its greedy argument assumes no later path can be shorter — valid only when all weights are non-negative. But the path $S \to B \to A$ costs $6 + (-5) = 1 < 2$. **Correct distances:** $d(S) = 0$, $d(B) = 6$, $d(A) = \mathbf{1}$.
Dijkstra reports $d(A) = 2$ — wrong. With negative edges one must use **Bellman–Ford** ($O(nm)$), which relaxes all edges $n-1$ times and also detects negative cycles.

Q22. Apply Kruskal's algorithm to the graph with edges AB=3, AC=1, BC=2, BD=5, CD=4, CE=6, DE=2. List accepted and rejected edges and give the total weight.

Solution. Sorted: AC 1, BC 2, DE 2, AB 3, CD 4, BD 5, CE 6.

Edge	Action	Reason
AC (1)	accept	$\{A, C\}$
BC (2)	accept	$\{A, B, C\}$
DE (2)	accept	$\{D, E\}$
AB (3)	**reject**	cycle A-C-B-A
CD (4)	accept	joins the components; all 5 vertices connected
BD (5)	**reject**	cycle
CE (6)	**reject**	cycle; 4 edges already chosen

MST edges: AC, BC, DE, CD. **Total weight** $= 1+2+2+4 = 9$. Four edges for five vertices $(n-1)$ ✓.

Q23. Apply Prim's algorithm starting from vertex A to the same graph, and confirm the total weight matches Kruskal's.

Solution.

Tree so far	Cheapest edge leaving it	Add
$\{A\}$	AC=1, AB=3	AC (1)
$\{A, C\}$	BC=2, CD=4, CE=6, AB=3	BC (2)
$\{A, B, C\}$	CD=4, BD=5, CE=6	CD (4)
$\{A, B, C, D\}$	DE=2, CE=6	DE (2)
$\{A, B, C, D, E\}$	—	stop

Total $= 1 + 2 + 4 + 2 = 9$ ✓— identical to Kruskal's, as the MST weight is unique even when the tree is not. Here the edge sets also coincide: $\{AC, BC, CD, DE\}$.

Q24. A network of 6 offices must be connected by cable. Costs (in lakhs): 12=4, 13=3, 23=1, 24=5, 34=6, 35=8, 45=2, 46=7, 56=9. Find the minimum total cost using Kruskal's algorithm.

Solution. Sorted: 23 1, 45 2, 13 3, 12 4, 24 5, 34 6, 46 7, 35 8, 56 9.

Edge	Action	Components
23 (1)	accept	$\{2, 3\}$
45 (2)	accept	$\{4, 5\}$
13 (3)	accept	$\{1, 2, 3\}$
12 (4)	**reject**	cycle
24 (5)	accept	$\{1, 2, 3, 4, 5\}$
34 (6)	**reject**	cycle
46 (7)	accept	all six connected — stop

MST edges: $23, 45, 13, 24, 46$ — five edges for six offices ✓. **Minimum total cost** $= 1 + 2 + 3 + 5 + 7 =$ **18 lakhs.**

Q25. For a graph where all edge weights are distinct, state whether the MST is unique, and justify.

Solution. **Yes, the MST is unique.**
Justification (cut property). Suppose two distinct MSTs T_1 and T_2 existed. Take the lightest edge e belonging to one but not the other — say $e \in T_1 \setminus T_2$. Adding e to T_2 creates exactly one cycle, which must contain an edge $f \notin T_1$. Since all weights are distinct and e was the lightest edge not shared, $w(e) < w(f)$. Then $T_2 - f + e$ is a spanning tree of strictly smaller weight than T_2, contradicting the minimality of T_2.
Hence no two distinct MSTs can exist when weights are distinct. (With repeated weights, ties can produce several MSTs of equal total weight.)

Q26. Explain the difference between the MST and the shortest-path tree, giving a triangle as a counterexample.

Solution. The **MST** minimises the *sum of all edge weights*; the **shortest-path tree** minimises the *path length from a fixed root to each individual vertex.* They answer different questions.
Counterexample. Triangle A, B, C with $AB = 2$, $BC = 2$, $AC = 3$.

- **MST:** the two cheapest edges AB and BC; total weight **4**.
- **Dijkstra's tree from** A**:** $d(B) = 2$ via AB; $d(C) = 3$ via the direct edge AC, since going through B would cost $2 + 2 = 4 > 3$. The tree is $\{AB, AC\}$; total weight **5**.

The trees differ. In the MST the route from A to C costs 4, worse than the true shortest distance of 3.

Q27. Give the time complexity of BFS, DFS, Dijkstra (binary heap), Kruskal and Prim, in terms of $n = |V|$ and $m = |E|$.

Solution.

Algorithm	Complexity	Reason
BFS	$\Theta(n+m)$	each vertex enqueued once, each edge scanned once
DFS	$\Theta(n+m)$	same accounting
Dijkstra (heap)	$O((n+m)\log n)$	n extract-mins and up to m decrease-keys, each $O(\log n)$
Kruskal	$O(m \log m)$	dominated by sorting the edges
Prim (heap)	$O(m \log n)$	like Dijkstra, keyed on edge weight

With a Fibonacci heap Dijkstra improves to $O(m + n \log n)$. Note $\log m = O(\log n)$ since $m \le n^2$, so Kruskal and Prim are comparable.

Q28. A graph is stored as an adjacency matrix. State the cost of (a) checking whether edge (u, v) exists, (b) listing all neighbours of u, (c) the space used. Compare with adjacency lists.

Solution.

Operation	Adjacency matrix	Adjacency list
(a) Edge existence (u, v)	$\Theta(1)$	$O(\deg u)$
(b) List neighbours of u	$\Theta(n)$	$\Theta(\deg u)$
(c) Space	$\Theta(n^2)$	$\Theta(n+m)$

Choice: use a matrix for *dense* graphs (m close to n^2) or when edge lookups dominate; use lists for *sparse* graphs, which is the usual case in practice (road networks, social graphs, web graphs). BFS and DFS achieve $\Theta(n+m)$ only with adjacency lists — with a matrix they degrade to $\Theta(n^2)$.

D. Graph Colouring (Q29–Q33)

Q29. Find the chromatic number of the complete graph K_5, of a tree with 10 vertices, of the cycle C_6 and of the cycle C_7.

Solution. $\chi(K_5) = \mathbf{5}$ — every pair is adjacent, so all five need distinct colours. $\chi(\text{tree}) = \mathbf{2}$ — trees have no cycles at all, hence no odd cycles, so they are bipartite (for $n \ge 2$). $\chi(C_6) = \mathbf{2}$ — an even cycle is bipartite; alternate two colours. $\chi(C_7) = \mathbf{3}$ — an odd cycle is not bipartite; two colours fail where the cycle closes.

Q30. Apply Welsh–Powell to the graph with edges $AB, AC, AD, BC, BD, CE, DE$. Give the colour classes and $\chi(G)$.

Solution. Degrees: A=3 (B, C, D); B=3 (A, C, D); C=3 (A, B, E); D=3 (A, B, E); E=2 (C, D). Sum = 14, $|E| = 7$ ✓.
Order (decreasing degree, alphabetical ties): A, B, C, D, E.

- **Colour 1:** A. B? adjacent — skip. C? adjacent — skip. D? adjacent — skip. E? not adjacent to A ✓. **Colour 1** $= \{A, E\}$.
- **Colour 2:** B. C? adjacent to B — skip. D? adjacent to B — skip. **Colour 2** $= \{B\}$.
- **Colour 3:** C. D? C and D are not adjacent ✓. **Colour 3** $= \{C, D\}$.

Three colours. Minimality: A, B, C are mutually adjacent (a triangle), so $\chi \ge 3$. Hence $\chi(G) = \mathbf{3}$. *Edge check:* AB(1,2), AC(1,3), AD(1,3), BC(2,3), BD(2,3), CE(3,1), DE(3,1) — all differ ✓.

Q31. Six examinations must be scheduled. Conflicts: E_1E_2, E_1E_3, E_2E_3, E_2E_4, E_3E_5, E_4E_5, E_4E_6, E_5E_6. Find the minimum number of slots and give a timetable.

Solution. Degrees: E_2=3 (E_1, E_3, E_4); E_3=3 (E_1, E_2, E_5); E_4=3 (E_2, E_5, E_6); E_5=3 (E_3, E_4, E_6); E_1=2; E_6=2. Sum = 16, $|E| = 8$ ✓.
Welsh–Powell order: $E_2, E_3, E_4, E_5, E_1, E_6$.

- **Slot 1:** E_2. E_3? adjacent — skip. E_4? adjacent — skip. E_5? not adjacent to E_2 ✓. E_1? adjacent to E_2 — skip. E_6? adjacent to E_5 — skip. **Slot 1** $= \{E_2, E_5\}$.
- **Slot 2:** E_3. E_4? E_3 and E_4 are not adjacent ✓. E_1? adjacent to E_3 — skip. E_6? adjacent to E_4 — skip. **Slot 2** $= \{E_3, E_4\}$.
- **Slot 3:** E_1. E_6? E_1 and E_6 are not adjacent ✓. **Slot 3** $= \{E_1, E_6\}$.

Minimum slots $= 3$. Necessary because E_1, E_2, E_3 form a triangle, so $\chi \geq 3$.

Slot	Examinations
1	E_2, E_5
2	E_3, E_4
3	E_1, E_6

Q32. State and justify the bounds $\omega(G) \leq \chi(G) \leq \Delta(G) + 1$ for a graph with maximum degree 5 containing a triangle.

Solution. **Lower bound $\chi \geq \omega$:** a clique of size ω has every pair adjacent, so its vertices require ω distinct colours among themselves. Here a triangle gives $\omega \geq 3$, so $\chi \geq \mathbf{3}$.
Upper bound $\chi \leq \Delta + 1$: colour the vertices greedily in any order. When a vertex is reached, at most Δ of its neighbours are already coloured, so among $\Delta + 1$ colours at least one is free. Here $\Delta = 5$, so $\chi \leq \mathbf{6}$.
Conclusion: $3 \leq \chi(G) \leq 6$. (Brooks' theorem sharpens this to $\chi \leq \Delta = 5$ unless G is a complete graph or an odd cycle.)

Q33. Explain how graph colouring models register allocation in a compiler, and state what a colouring failure means in that context.

Solution. **The model.** Each program variable becomes a *vertex*. An *edge* joins two variables whose live ranges overlap — i.e. both hold a value at the same point in the program, so they cannot share a register. The available CPU registers are the *colours*. A proper colouring assigns registers so that no two simultaneously-live variables share one.
Colouring failure. If the interference graph needs more colours than the machine has registers ($\chi(G) > k$), the allocation is impossible as posed. The compiler then **spills** one or more variables to memory — removing that vertex from the graph and replacing its uses with load/store instructions — and re-attempts the colouring. Spilling is correct but slow, so compilers choose spill candidates carefully (typically a variable with high degree and low use frequency).
Since optimal colouring is NP-complete, real compilers use heuristics (Chaitin's algorithm: repeatedly remove a vertex of degree $< k$, colour the rest, then reinsert).

E. Network Flow (Q34–Q40)

Q34. Define a flow network, a valid flow, and the value of a flow. State the two constraints a flow must satisfy.

Solution. A **flow network** is a directed graph with a distinguished *source* s, a *sink* t, and a non-negative *capacity* $c(u, v)$ on each arc.
A **flow** f assigns a value to each arc subject to two constraints:

1. **Capacity constraint:** $0 \le f(u,v) \le c(u,v)$ — no arc carries more than it can.
2. **Flow conservation:** for every vertex $v \notin \{s,t\}$, total flow in = total flow out — nothing is created or destroyed except at s and t.

The **value** $|f|$ is the net flow out of the source (equivalently, into the sink).

Q35. For the network $s \to a$ (10), $s \to b$ (5), $a \to b$ (4), $a \to t$ (7), $b \to t$ (10), find the maximum flow by Ford–Fulkerson, showing each augmenting path.

Solution.

#	Augmenting path	Bottleneck	Running flow
1	$s \to a \to t$	$\min(10,7) = 7$	7
2	$s \to b \to t$	$\min(5,10) = 5$	12
3	$s \to a \to b \to t$	$\min(3,4,5) = 3$	**15**

After path 1: sa has 3 left, at saturated. After path 2: sb saturated, bt has 5 left. After path 3: sa saturated, ab has 1 left, bt has 2 left.
Residual search: from s, both $s \to a$ and $s \to b$ are saturated. No vertex other than s is reachable, so **no augmenting path remains**.
Maximum flow $= 15$**.**

Q36. For the network of the previous question, identify the minimum cut and verify the max-flow min-cut theorem.

Solution. Reachable from s in the final residual network: only s itself (both outgoing arcs are saturated).

$$S = \{s\}, \qquad T = \{a,b,t\}.$$

Arcs from S to T: $s \to a$ (capacity 10) and $s \to b$ (capacity 5).

$$c(S,T) = 10 + 5 = \mathbf{15} = \text{max flow } \checkmark$$

The theorem is verified. Both crossing arcs are saturated — the signature of a minimum cut. Here the source's own outgoing capacity is the bottleneck, so no improvement anywhere downstream could raise throughput.

Q37. In the network $s \to a$ (6), $s \to b$ (4), $a \to c$ (3), $a \to d$ (4), $b \to d$ (5), $c \to t$ (5), $d \to t$ (6), find the maximum flow and the minimum cut.

Solution.

#	Path	Bottleneck	Flow
1	$s \to a \to c \to t$	$\min(6,3,5) = 3$	3
2	$s \to a \to d \to t$	$\min(3,4,6) = 3$	6
3	$s \to b \to d \to t$	$\min(4,5,3) = 3$	**9**

After path 3, dt carries $3 + 3 = 6$ and is saturated; sa is saturated ($3 + 3 = 6$).
Residual search: from s only $s \to b$ has capacity (1). From b: $b \to d$ has 2 left, so d is reachable. From d: $d \to t$ saturated; reverse arc $d \to a$ (residual 3) makes a reachable. From a: $a \to c$ has 0 left; reverse $a \to s$. So c and t are unreachable — no augmenting path.
Maximum flow $= 9$**.**
Minimum cut: $S = \{s,b,d,a\}$, $T = \{c,t\}$. Arcs $S \to T$: $a \to c$ (3) and $d \to t$ (6).

$$c(S,T) = 3 + 6 = \mathbf{9} = \text{max flow } \checkmark$$

Q38. Explain why reverse (residual) arcs are necessary in Ford–Fulkerson, using a concrete example where a greedy first choice would otherwise block the optimum.

Solution. **The problem.** A greedy augmenting path can commit flow to an arc that a better solution needs for something else. Without a way to *undo* that commitment the algorithm would stop short of the maximum.
Concrete example. Network: $s \to a$ (1), $s \to b$ (1), $a \to b$ (1), $a \to t$ (1), $b \to t$ (1).

- Greedy first path $s \to a \to b \to t$ (bottleneck 1) gives flow 1. Now sa, ab and bt are all saturated.
- Forward-only search: $s \to b$ has capacity, but $b \to t$ is saturated, so no further path is found. The algorithm would report **1**.
- **With the reverse arc** $b \to a$ (residual 1, since $f(a,b) = 1$): the path $s \to b \xrightarrow{\text{rev}} a \to t$ has bottleneck 1. Pushing along it *cancels* the flow on $a \to b$ and reroutes it. Total flow becomes **2**.

Conclusion. The true maximum is 2 (both $s \to a \to t$ and $s \to b \to t$ carry 1 unit). Reverse arcs let a later augmentation retract an earlier, suboptimal routing decision, which is exactly what makes Ford–Fulkerson provably optimal rather than merely greedy.

Q39. Four students $S_1, \ldots, S_4$ must be assigned to four projects $P_1, \ldots, P_4$. S_1 can take P_1, P_3; S_2 can take P_1, P_2; S_3 can take P_2, P_3; S_4 can take P_4. Model this as a max-flow problem and find the maximum matching.

Solution. **Model.** Add a source s with a capacity-1 arc to each student, a capacity-1 arc from each student to every project they can take, and a capacity-1 arc from each project to a sink t. The unit capacities enforce "one project per student, one student per project", and the max flow equals the maximum matching size.
Augmenting paths:

#	Path	Flow
1	$s \to S_1 \to P_1 \to t$	1
2	$s \to S_2 \to P_2 \to t$	2
3	$s \to S_3 \to P_3 \to t$	3
4	$s \to S_4 \to P_4 \to t$	**4**

Maximum matching = 4: S_1–P_1, S_2–P_2, S_3–P_3, S_4–P_4. A complete (perfect) assignment exists — every student gets a project.

Q40. Repeat the previous question with S_1 able to take only P_1, S_2 only P_1, S_3 taking P_1, P_2, and S_4 taking P_4. Find the maximum matching and explain why a complete assignment is impossible.

Solution. **Augmenting paths:** $s \to S_1 \to P_1 \to t$ (flow 1); $s \to S_3 \to P_2 \to t$ (flow 2); $s \to S_4 \to P_4 \to t$ (flow 3). S_2's only option P_1 is already taken, and the reverse arc $P_1 \to S_1$ leads to a student with no other option. No augmenting path exists.
Maximum matching = 3: S_1–P_1, S_3–P_2, S_4–P_4. S_2 is left unassigned.
Why complete assignment is impossible (Hall's condition). Consider the set of students $\{S_1, S_2\}$. Between them they are qualified for only *one* project, P_1. Two students cannot be assigned to one project, so no matching can cover both. Hall's marriage theorem states a perfect matching exists iff every set of students W collectively qualifies for at least $|W|$ projects — violated here since $|W| = 2$ but $|N(W)| = 1$.
The minimum cut confirms the value 3, and P_3 receives nobody since no student listed it.